Persistencia con JPA / Hibernate

ORM. Configuración de JPA. Relaciones con anotaciones. Lazy y eager. JPQL.

```java
@Entity
@Table(name = "cursos")
public class Curso {
   @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @Column(nullable = false, length = 120)
   private String nombre;

   @OneToMany(mappedBy = "curso", fetch = FetchType.LAZY,
        cascade = CascadeType.ALL, orphanRemoval = true)
   private List<Leccion> lecciones = new ArrayList<>();
}

/* JPQL */
SELECT c FROM Curso c JOIN c.lecciones l WHERE c.nombre = :nombre
```







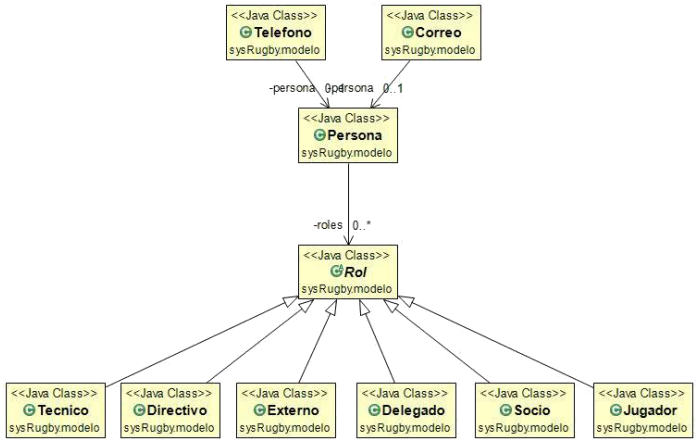




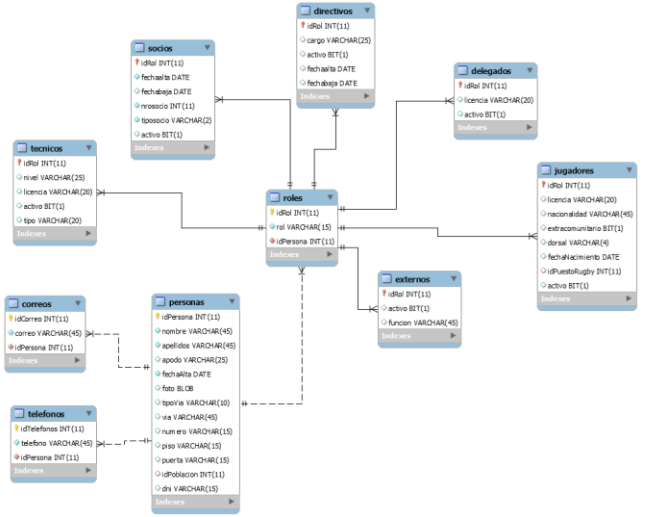
ORM | CONFIGURACIÓN JPA | ONE TO ONE | ONE TO MANY | MANY TO ONE | MANY TO MANY | LAZY / EAGER | JPQL

Introducción

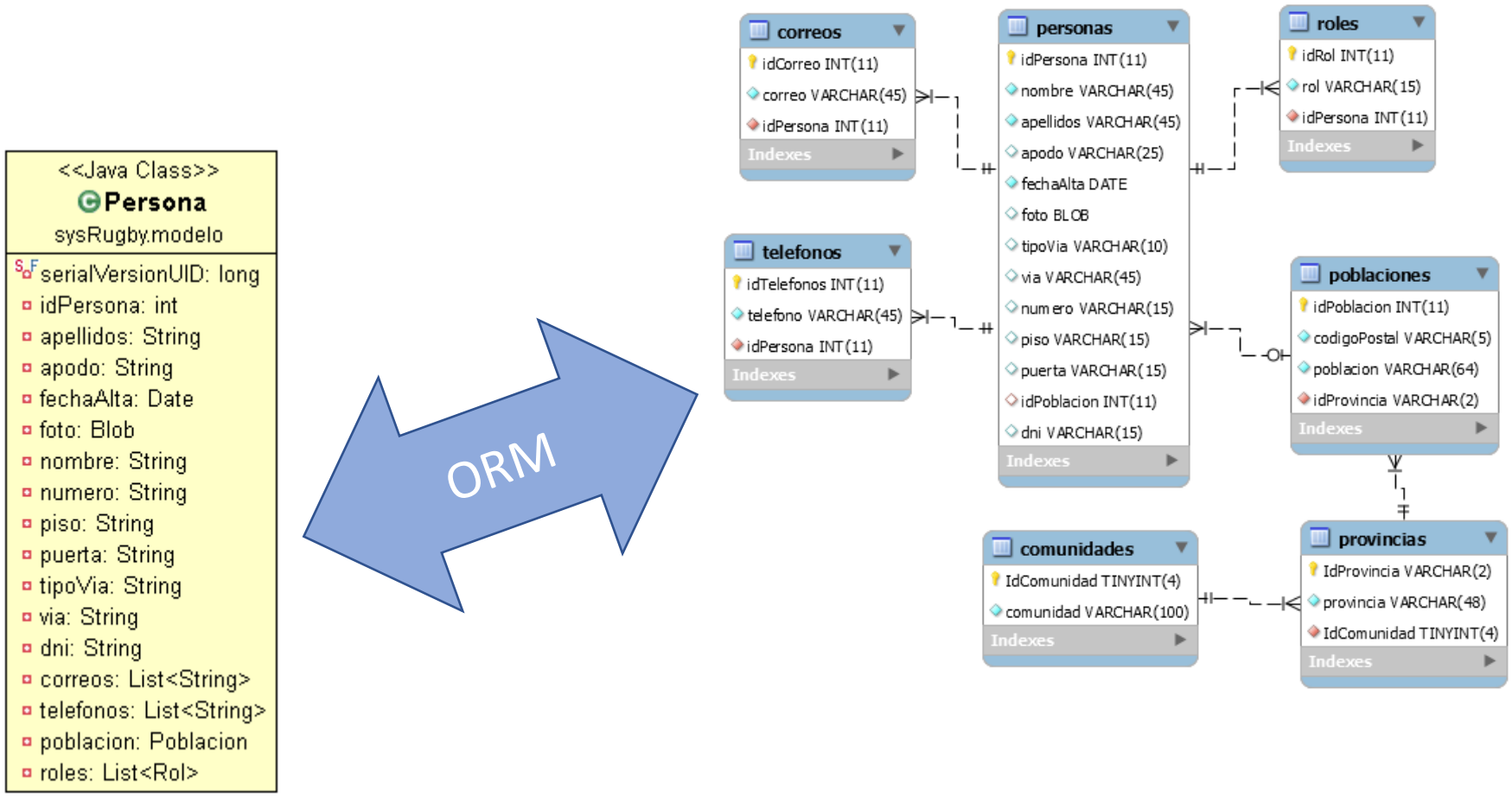
OBJETOS
- Atributos (complejos)
- Métodos
- Herencia



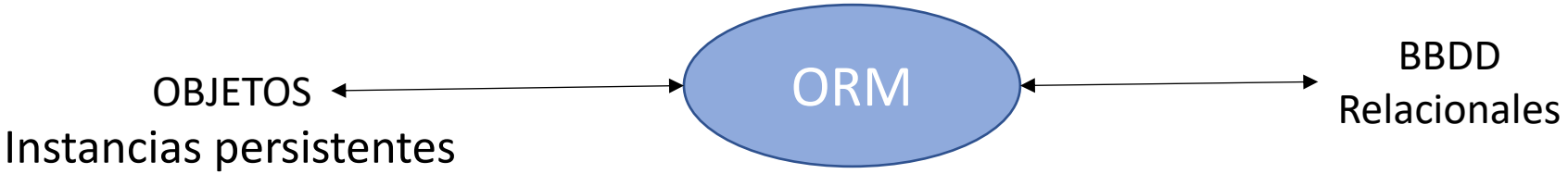
BBDD Relacionales
- Esquemas
- Tablas
  - Filas
  - PK
  - FK
- Indices



ORM


<<Java Class>>
Persona
sysRugby.modelo
serialVersionUID: long
idPersona: int
apellidos: String
apodo: String
fechaAlta: Date
foto: Blob
nombre: String
numero: String
piso: String
puerta: String
tipoVia: String
via: String
dni: String
correos: List<String>
telefonos: List<String>
poblacion: Poblacion
roles: List<Rol>
ORM
correos
idCorreo INT(11)
correo VARCHAR(45)
idPersona INT(11)
Indexes
personas
idPersona INT(11)
nombre VARCHAR(45)
apellidos VARCHAR(45)
apodo VARCHAR(25)
fechaAlta DATE
foto BLOB
tipoVia VARCHAR(10)
via VARCHAR(45)
numero VARCHAR(15)
piso VARCHAR(15)
puerta VARCHAR(15)
idPoblacion INT(11)
dni VARCHAR(15)
Indexes
roles
idRol INT(11)
rol VARCHAR(15)
idPersona INT(11)
Indexes
telefonos
idTelefonos INT(11)
telefono VARCHAR(45)
idPersona INT(11)
Indexes
poblaciones
idPoblacion INT(11)
codigoPostal VARCHAR(5)
poblacion VARCHAR(64)
idProvincia VARCHAR(2)
Indexes
comunidades
IdComunidad TINYINT(4)
comunidad VARCHAR(100)
Indexes
provincias
IdProvincia VARCHAR(2)
provincia VARCHAR(48)
IdComunidad TINYINT(4)
Indexes

ORM - Object Relational Mapping



1999	EJB 1.0
2001	EJB 2.0
2002	JDO
2002	Hibernate
2006	JPA 1.0 (EJB 3.0) – JAVA EE5
2009	JPA 2.0 – JAVA EE6
2013	JPA 2.1 – JAVA EE7
2017	JPA 2.2
2020	Jakarta persistence 3.0
2024	Jakarta Persistence 3.2

Hibernate / JPA

JPA es un estándar de Java que define un API para los ORM en Java. Es una especificación.

Los entity beans de los servidores Java EE se declaran siguiendo el api JPA.

HIBERNATE es un ORM de RedHat, creado por Gavin King, luego JBoss contrató al grupo de desarrollo y dio soporte al proyecto.

Implementa el estándar JPA

IMPLEMENTACIONES JPA	
Hibernate	RedHat
Eclipselink	Eclipse Foundation
Toplink	Oracle
ObjectDB	
CocoBase	
OpenJPA	Apache
Kodo	Bea

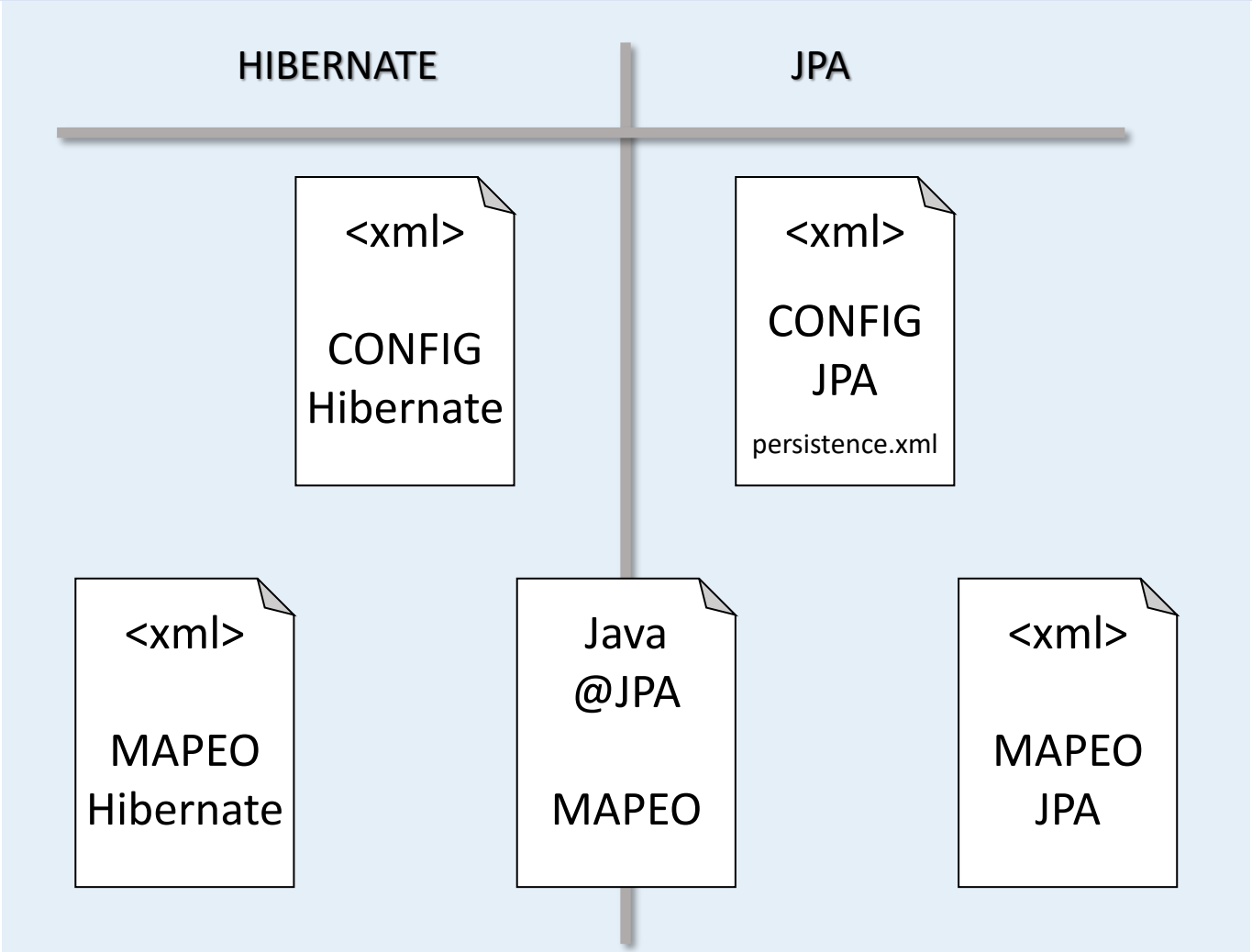
Características

- Las Entity class deben cumplir el estándar JavaBean
  - Son clases simples, no necesitan heredar de otra clase, sólo implementa Serializable
  - Constructor por defecto (explícito o implícito)
  - Debe ser de primer nivel (no interna)
  - No ser final
  - Implementa Serializable
  - Getter y Setter para sus atributos
- Configuración
  - Archivo de Configuración XML (HIBERNATE o JPA)
- Mapeo (Metadatos):
  - Anotaciones (Utiliza el estándar JPA)
  - Archivo de Mapeo XML (HIBERNATE o JPA)

JPA / Hibernate

Configuración

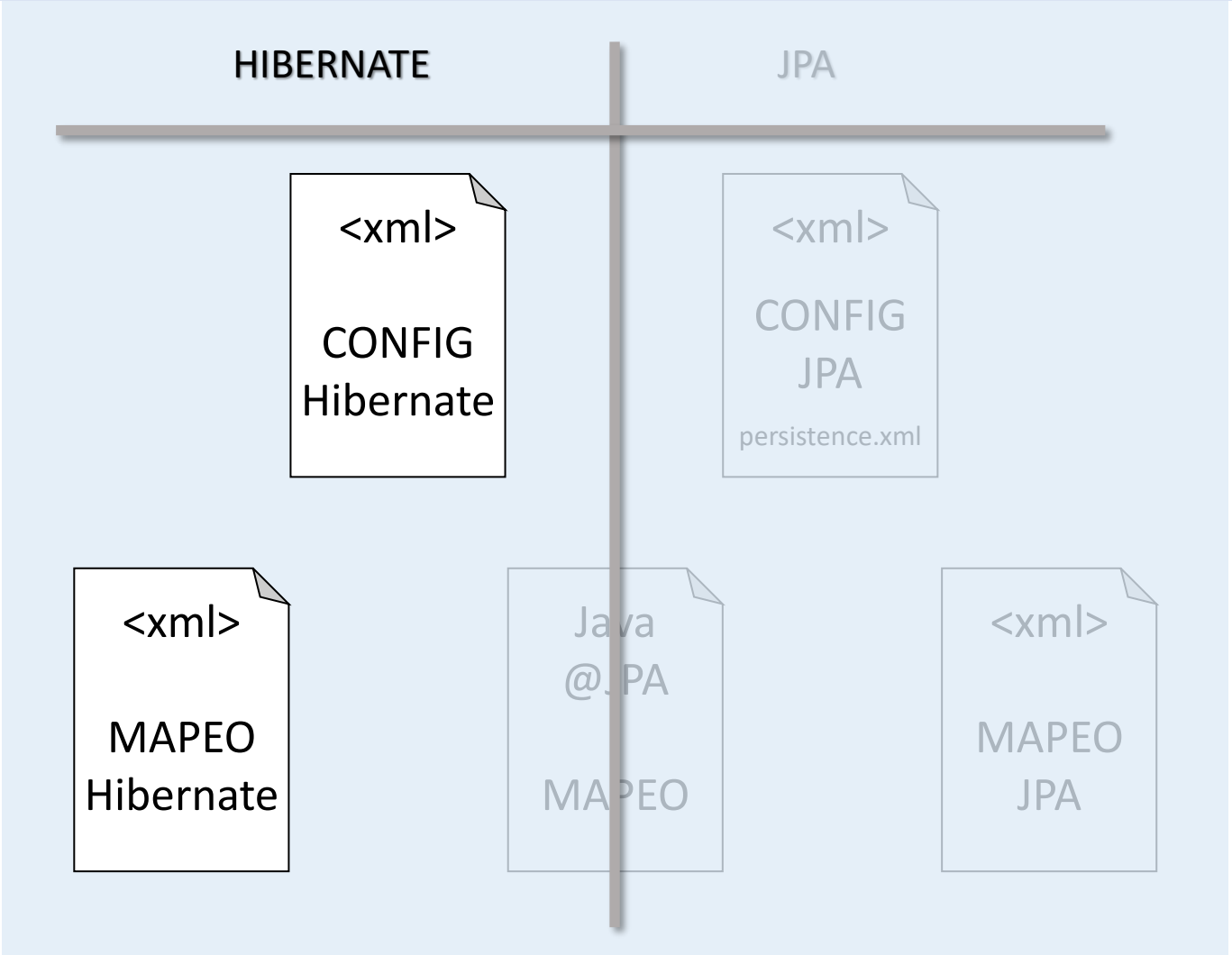
Mapeo



Hibernate – Mapeo xml

Configuración

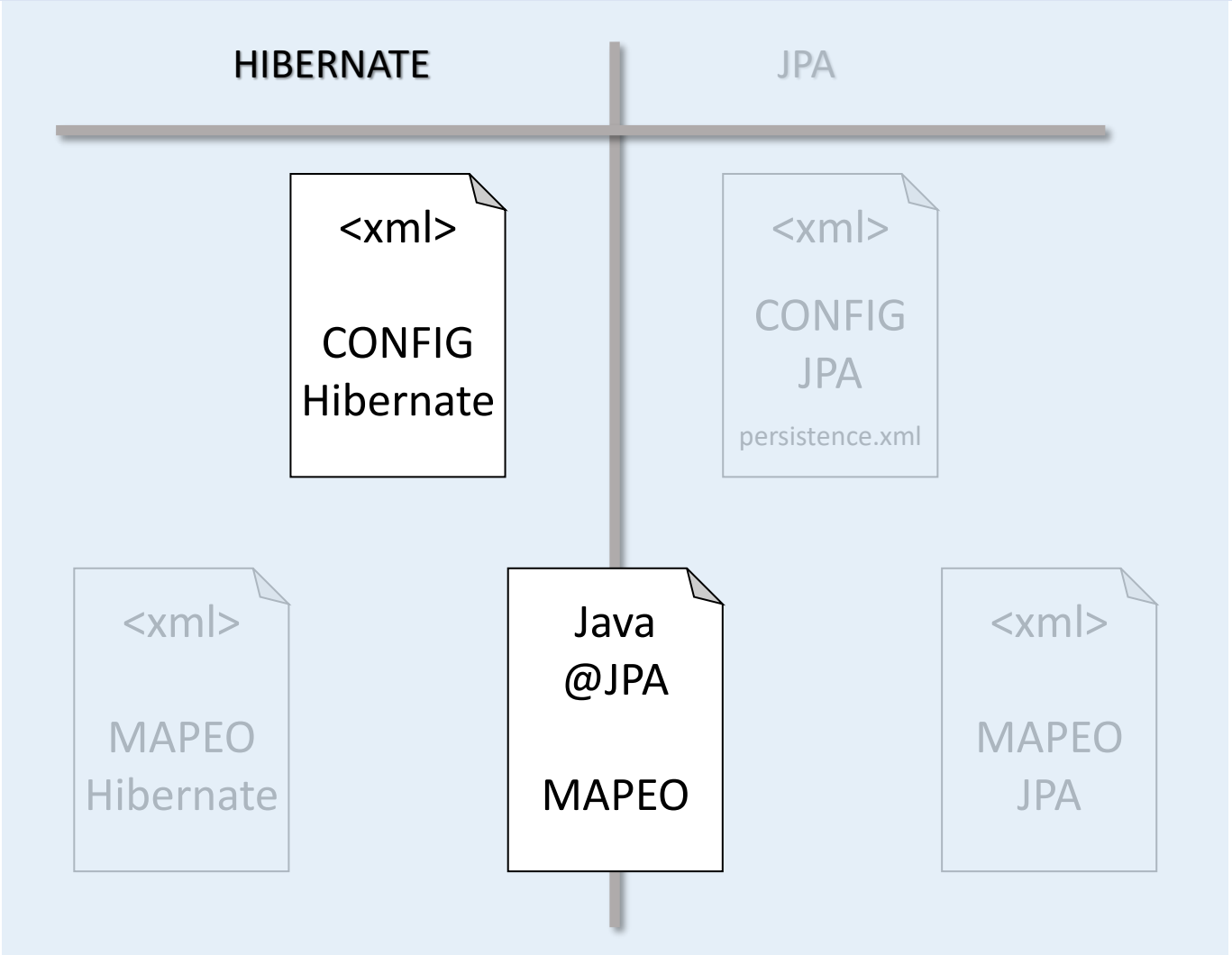
Mapeo



HIBERNATE – Anotaciones JPA

Configuración

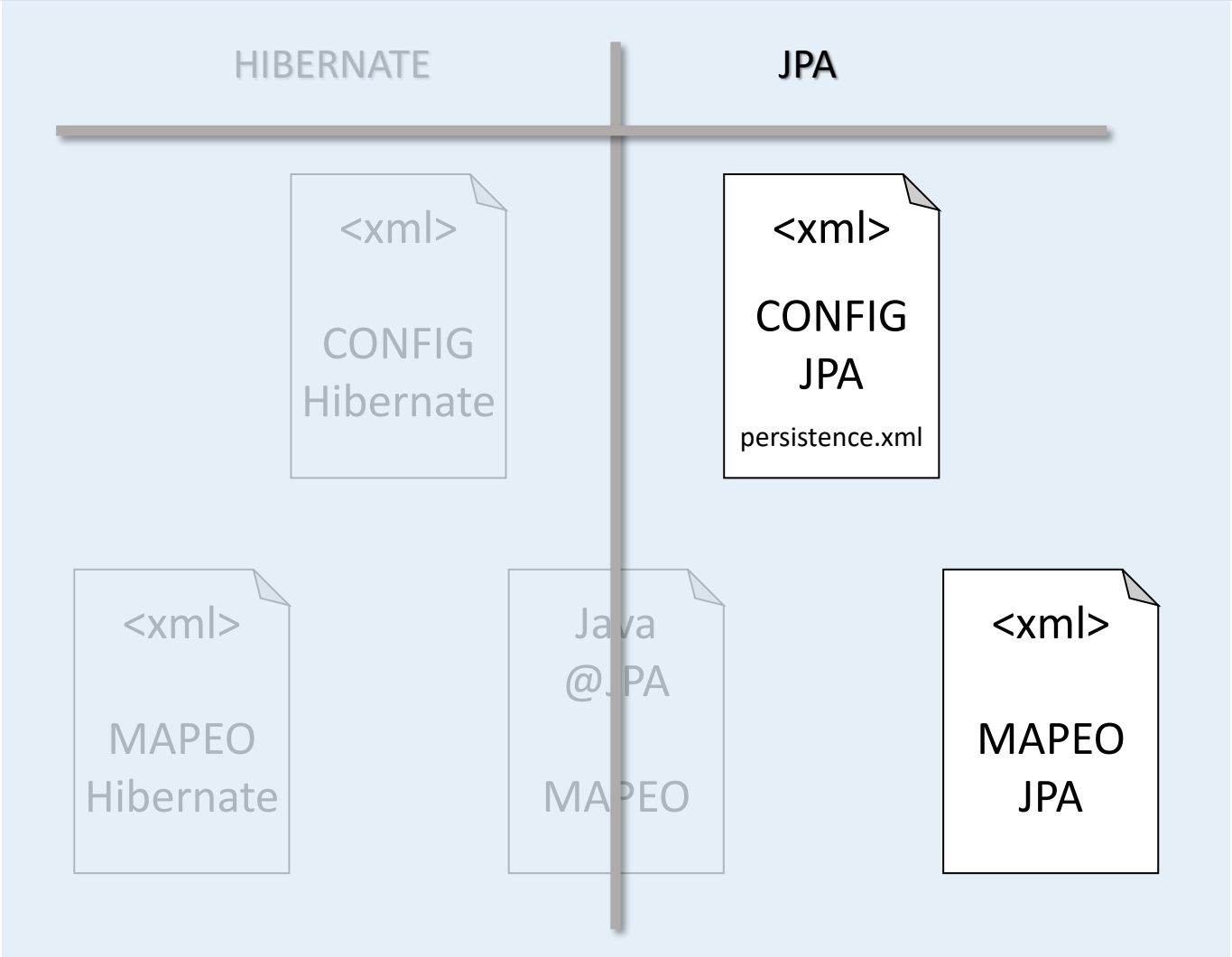
Mapeo



JPA – Mapeo xml

Configuración

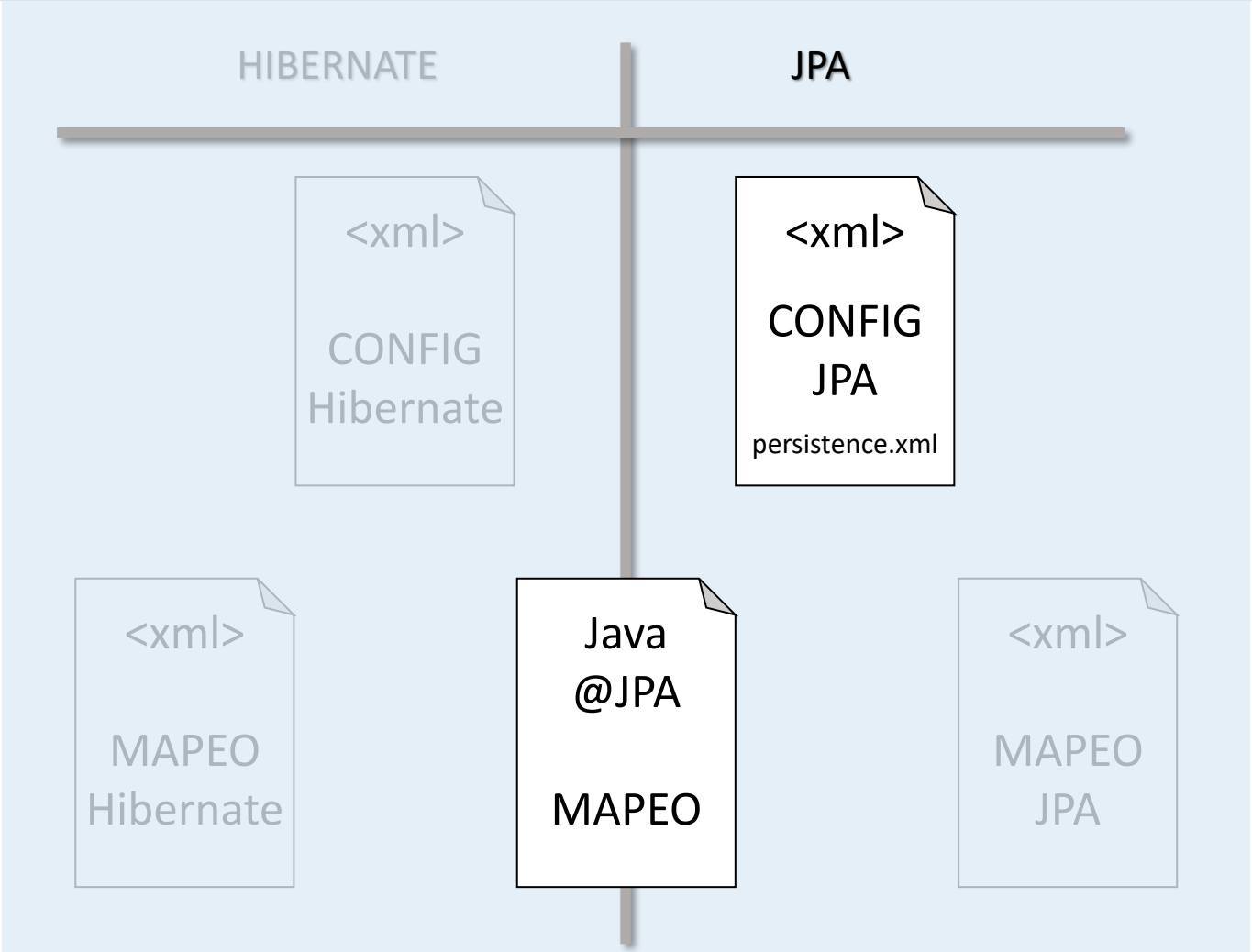
Mapeo



JPA – Anotaciones JPA

Configuración

Mapeo



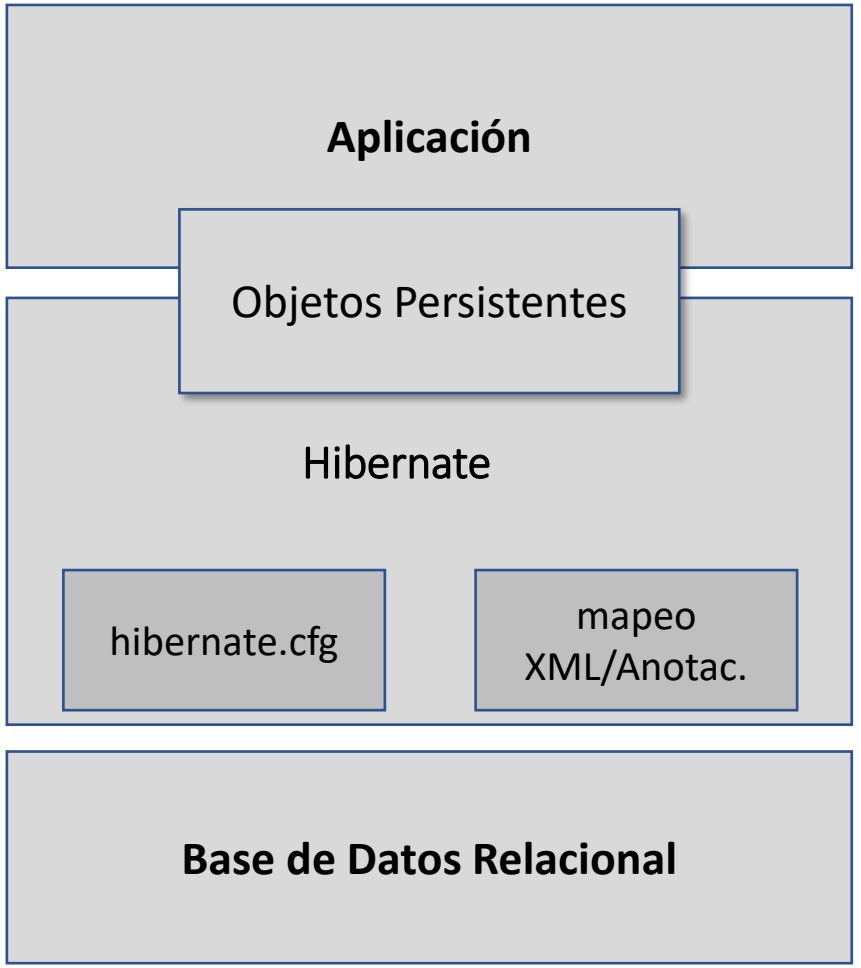
Mapeo

ANOTACIONES	
+	Todo en el mismo lugar
-	Es necesario recompilar
-	Menos portable (si no se quiere utilizar JPA)

DESCRIPTOR XML	
-	Mantenimiento de ficheros externos (1 x clase)
+	No es necesario recompilar

Si existen ambos, prevalece el Descriptor XML

Hibernate


Aplicación
Objetos Persistentes
Hibernate
hibernate.cfg
mapeo
XML/Anotac.
Base de Datos Relacional

Hibernate – hibernate.cfg.xml

En la aplicación se debe incluir el archivo hibernate.cfg.xml donde se configura Hibernate.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>

  <session-factory>
    <property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
    <property name="hibernate.show_sql">true</property>

    <property name="connection.driver_class">com.mysql.jdbc.Driver</property>
    <property name="connection.url">jdbc:mysql://localhost:3306/curso_hibernate</property>
    <property name="connection.username">root</property>
    <property name="connection.password">root</property>

    <mapping resource="persist-xml/introduccion/persona.hbm.xml"/>
  </session-factory>

</hibernate-configuration>
```

JPA - persistence.xml

En la carpeta **META-INF** se debe incluir el archivo persistence.xml con la configuración de JPA. Ej. mapeo con ficheros xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
         http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd" version="2.0">

  <persistence-unit name="UnidadAgenda" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

    <mapping-file>persist-xml/introduccion/Persona.xml</mapping-file>

    <properties>
      <property name="hibernate.show_sql" value="true" />
      <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL5Dialect" />
      <property name="hibernate.connection.pool_size" value="1" />

      <property name="hibernate.connection.driver_class" value="com.mysql.jdbc.Driver"/>
     <property name="hibernate.connection.url" value="jdbc:mysql://localhost/curso_hibernate"/>
      <property name="hibernate.connection.username" value="root"/>
      <property name="hibernate.connection.password" value="root"/>
    </properties>
  </persistence-unit>

</persistence>
```

JPA - persistence.xml

En la carpeta **META-INF** se debe incluir el archivo persistence.xml con la configuración de JPA. Ej. mapeo con anotaciones

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
         http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd" version="2.0">

  <persistence-unit name="UnidadAgenda" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

    <class>introduccion.Persona</class>

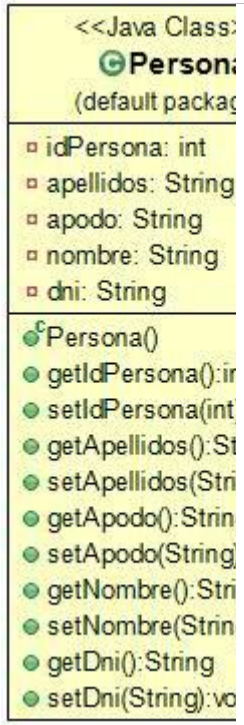
    <properties>
      <property name="hibernate.show_sql" value="true" />
      <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL5Dialect" />
      <property name="hibernate.connection.pool_size" value="1" />

      <property name="hibernate.connection.driver_class" value="com.mysql.jdbc.Driver"/>
   <property name="hibernate.connection.url" value="jdbc:mysql://localhost/curso_hibernate"/>
      <property name="hibernate.connection.username" value="root"/>
      <property name="hibernate.connection.password" value="root"/>
    </properties>
  </persistence-unit>

</persistence>
```

Hibernate – Mapeo xml

<hibernate-mapping> <class> <id> <property>
Ej. persona.hbm.xml



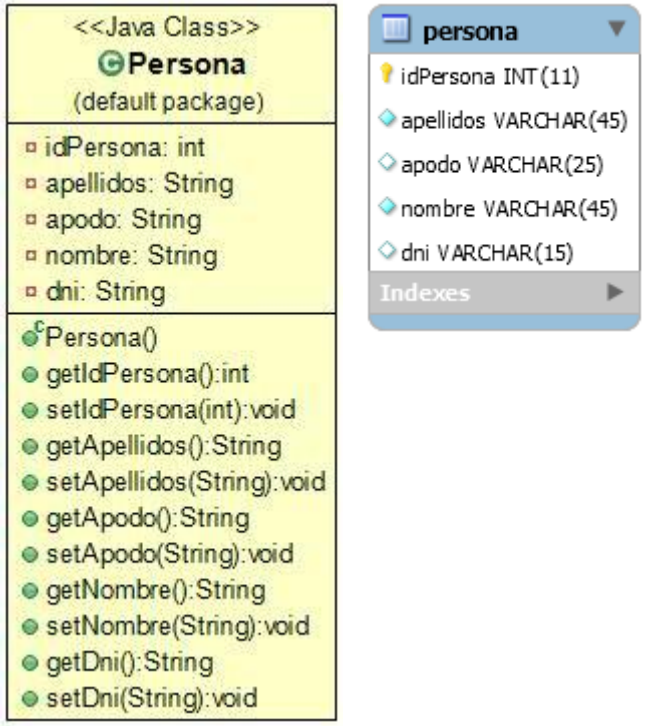

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC "-
//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-
mapping-3.0.dtd">

<hibernate-mapping>
  <class name="introduccion.Persona">
    <id name="idPersona" type="integer" />
    <property name="apellidos" />
    <property name="apodo" />
    <property name="nombre" />
    <property name="dni" />
  </class>
</hibernate-mapping>
```

Todas las propiedades serán mapeadas a una tabla con el mismo nombre de la clase y cada propiedad a una columna homónima

HIBERNATE – Mapeo xml

<hibernate-mapping> <class> <id> <property>
Ej. persona.hbm.xml




```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC "-
//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-
mapping-3.0.dtd">

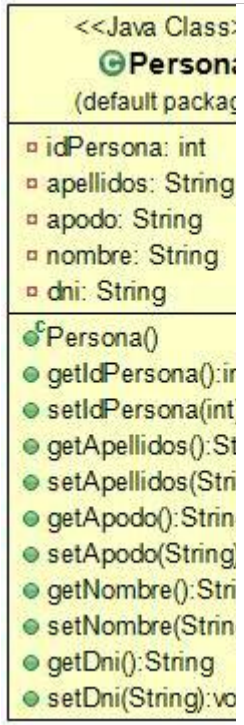
<hibernate-mapping>
  <class name="introduccion.Persona">
    <id name="idPersona" type="integer" />
    <property name="apellidos" />
    <property name="apodo" />
    <property name="nombre" />
    <property name="dni" />
  </class>
</hibernate-mapping>
```

Todas las propiedades serán mapeadas a una tabla con el mismo nombre de la clase y cada propiedad a una columna homónima

JPA - @Entity / @Id

@Entity Refiere a la clase e indica que será una entidad

@Id Indica el atributo que se corresponde con la PK




```java
import java.io.Serializable;
import javax.persistence.*;

@Entity
public class Persona implements Serializable {
@Id
private int idPersona;
private String apellidos;
private String apodo;
private String nombre;
private String dni;

public Persona() {
}

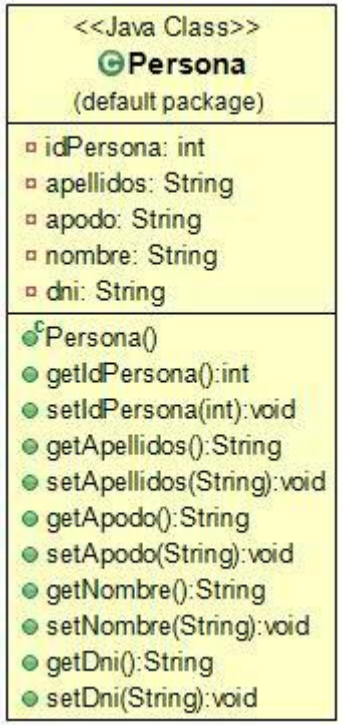
public int getIdPersona() {
return idPersona;
}
…
```

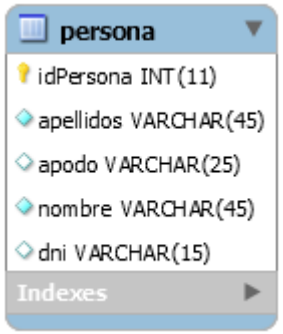
Todas las propiedades serán mapeadas a una tabla con el mismo nombre de la clase y cada propiedad a una columna homónima

JPA - @Entity / @Id

@Entity Refiere a la clase e indica que será una entidad

@Id Indica el atributo que se corresponde con la PK






```java
import java.io.Serializable;
import javax.persistence.*;

@Entity
public class Persona implements Serializable {
@Id
private int idPersona;
private String apellidos;
private String apodo;
private String nombre;
private String dni;

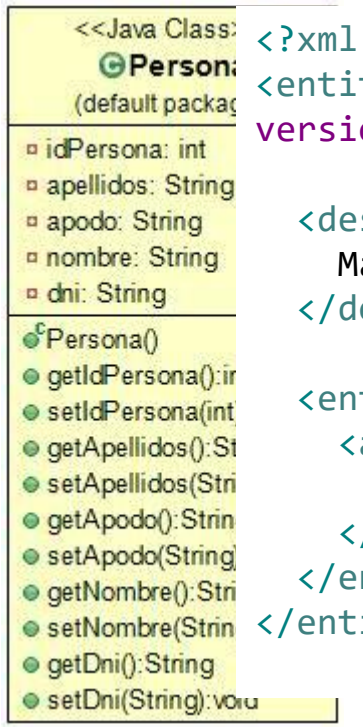
public Persona() {
}

public int getIdPersona() {
return idPersona;
}
…
```

Todas las propiedades serán mapeadas a una tabla con el mismo nombre de la clase y cada propiedad a una columna homónima

JPA – Mapeo xml

<entity> <attributes> <id>
Ej. Persona.xml




```xml
<?xml version="1.0" encoding="UTF-8"?>
<entity-mappings xmlns="http://java.su . . .
version="2.0">

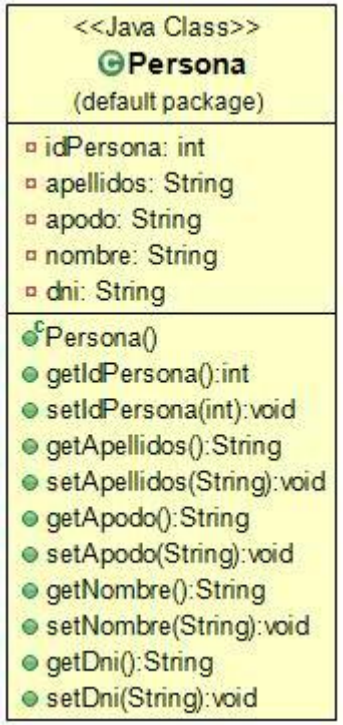
  <description>
    Mapeo minimo de una Entidad persistente
  </description>

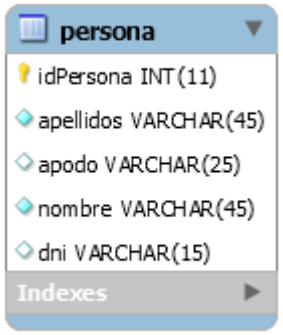
  <entity class="jpa.Persona">
    <attributes>
      <id name="idPersona" />
    </attributes>
  </entity>
</entity-mappings>
```

Todas las propiedades serán mapeadas a una tabla con el mismo nombre de la clase y cada propiedad a una columna homónima

JPA – Mapeo xml

<entity> <attributes> <id>
Ej. Persona.xml






```
<?xml version="1.0" encoding="UTF-8"?>
<entity-mappings xmlns="http://java.su . . .
version="2.0">

  <description>
    Mapeo minimo de una Entidad persistente
  </description>

  <entity class="jpa.Persona“>
    <attributes>
      <id name="idPersona" />
    </attributes>
  </entity>
</entity-mappings>
```

Todas las propiedades serán mapeadas a una tabla con el mismo nombre de la clase y cada propiedad a una columna homónima

Hibernate - Session

```
SessionFactory sessionFactory;
Session session;
Configuration configuration = new Configuration();
configuration.configure("persist-xml/hibernate.cfg.xml");

ServiceRegistry serviceRegistry;
serviceRegistry = new StandardServiceRegistryBuilder()…

sessionFactory =
configuration.buildSessionFactory(serviceRegistry);

session=sessionFactory.openSession();
. . .
session.close();
sessionFactory.close();
```

- **SessionFactory**
- **Session**

- **ServiceRegistry**
  - **Se obtiene de StandardServiceRegistryBuilder()**
- **Configuration**
  - **.configure(String xml)**
  - ***SessionFactory .buildSessionFactory(ServiceRegistry sr)***
- ***SessionFactory***
  - ***Session .openSession()***

Hibernate - Session

- id **save**(Object o)
- void **update**(Object o)
- void **saveOrUpdate**(Object o)
- void **delete**(Object o)
- Object **get**(Class c , Object pk)
- Crear Queries (vs. métodos)
- void **flush**()
- Connection **close**()

```
session=sessionFactory.openSession();

Persona p = new Persona();
p.setApellidos("...");

session.beginTransaction();
session.save(p);
session.getTransaction().commit();

Persona otra = (Persona)session.get(Persona.class, 27);
System.out.println(otra.getApodo());

session.close();
sessionFactory.close();
```

JPA - Entity Manager

```
EntityManagerFactory emf;
EntityManager em;

emf =
Persistence.createEntityManagerFactory("ejemplos_curso");
em = emf.createEntityManager();

em.close();
emf.close();
```

- **EntityManagerFactory**
- **EntityManager**

- **Persistence**
  - ***.createEntityManagerFactory(persistent_unit) EntityManagerFactory***
- **EntityManagerFactory**
  - **.createEntityManager() EntityManager**

JPA - EntityManager

- void **persist**(Objetc o)
- <T> **merge**(<T> o)
- void **remove**(Object o)
- <T> **find**(Class<T> , Object pk)
- EntityTransaction **getTransaction**()
- Crear Queries (vs. métodos)
- void **flush**()
- void **close**()

```
em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();

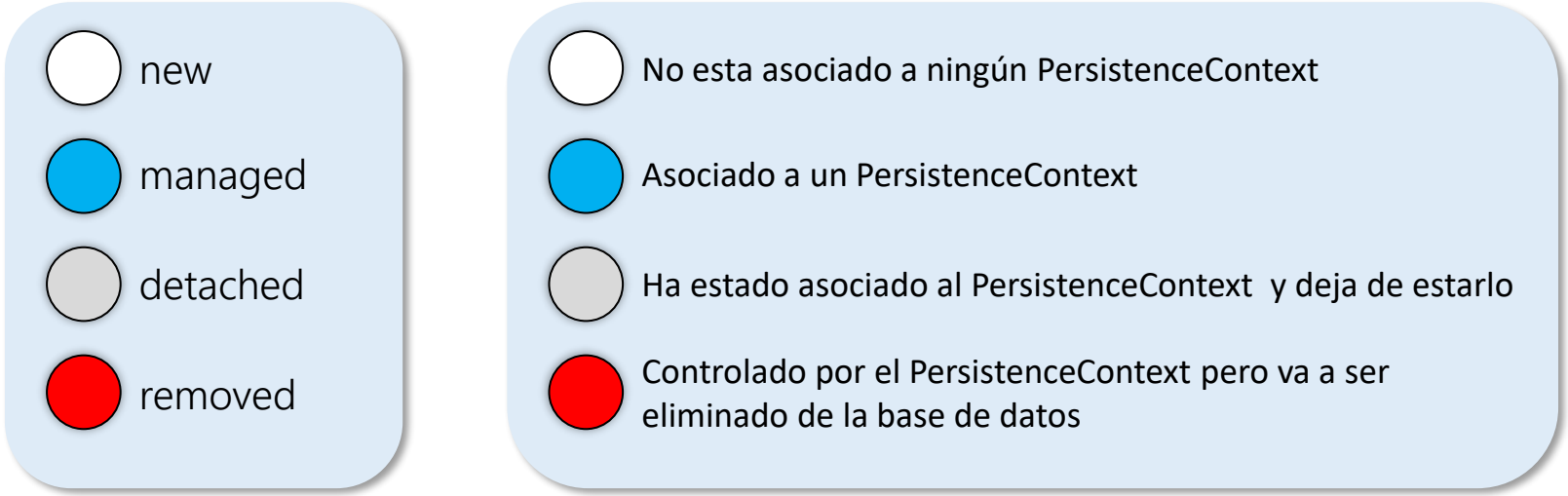
Persona p = new Persona();
p.setApellidos("...");

tx.begin();
em.persist(p);
tx.commit();

Persona otra = em.find(Persona.class, 27);
System.out.println(otra.getApodo());

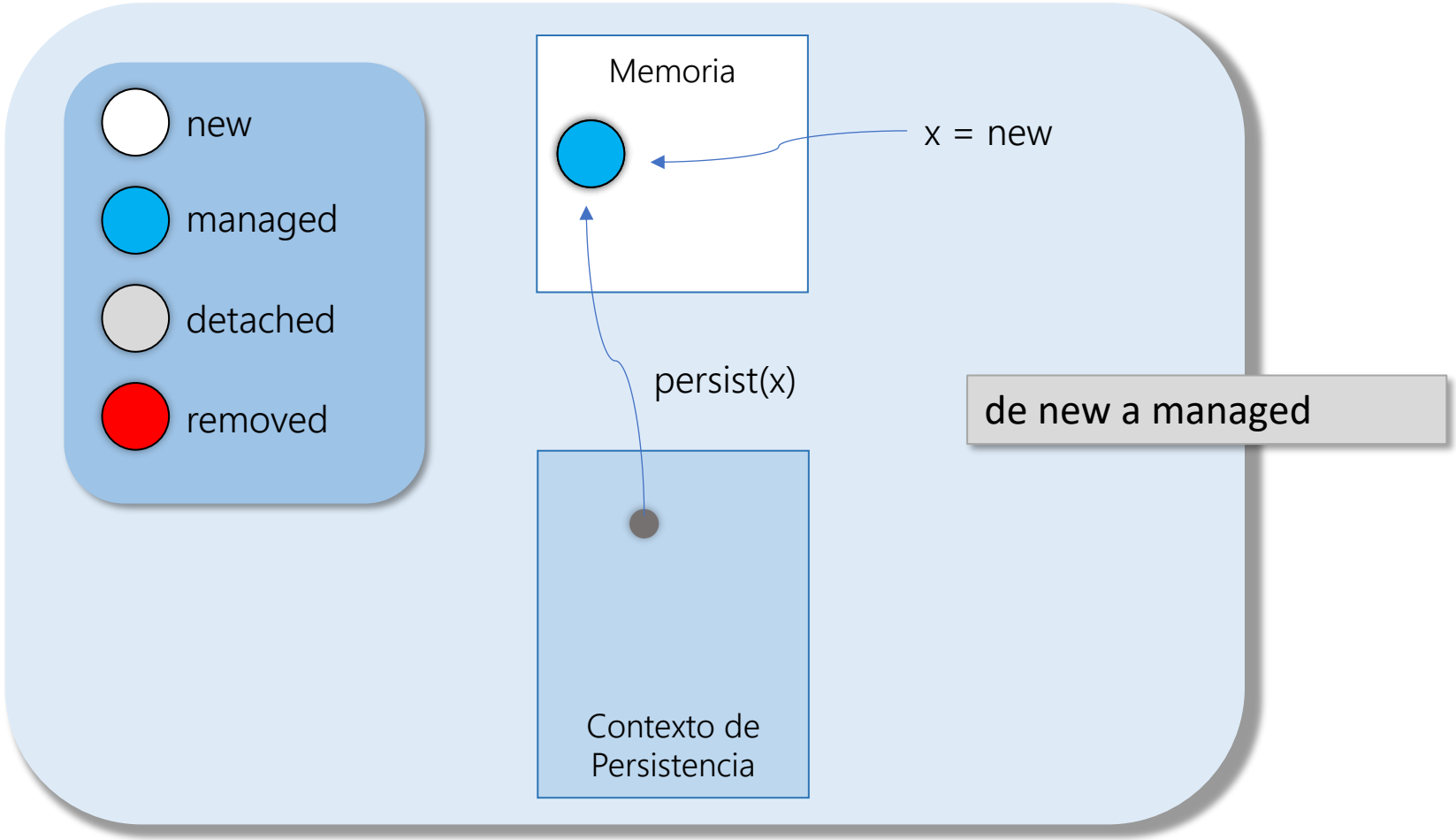
em.close();
emf.close();
```

Contexto de Persistencia - Estado de las Entidades



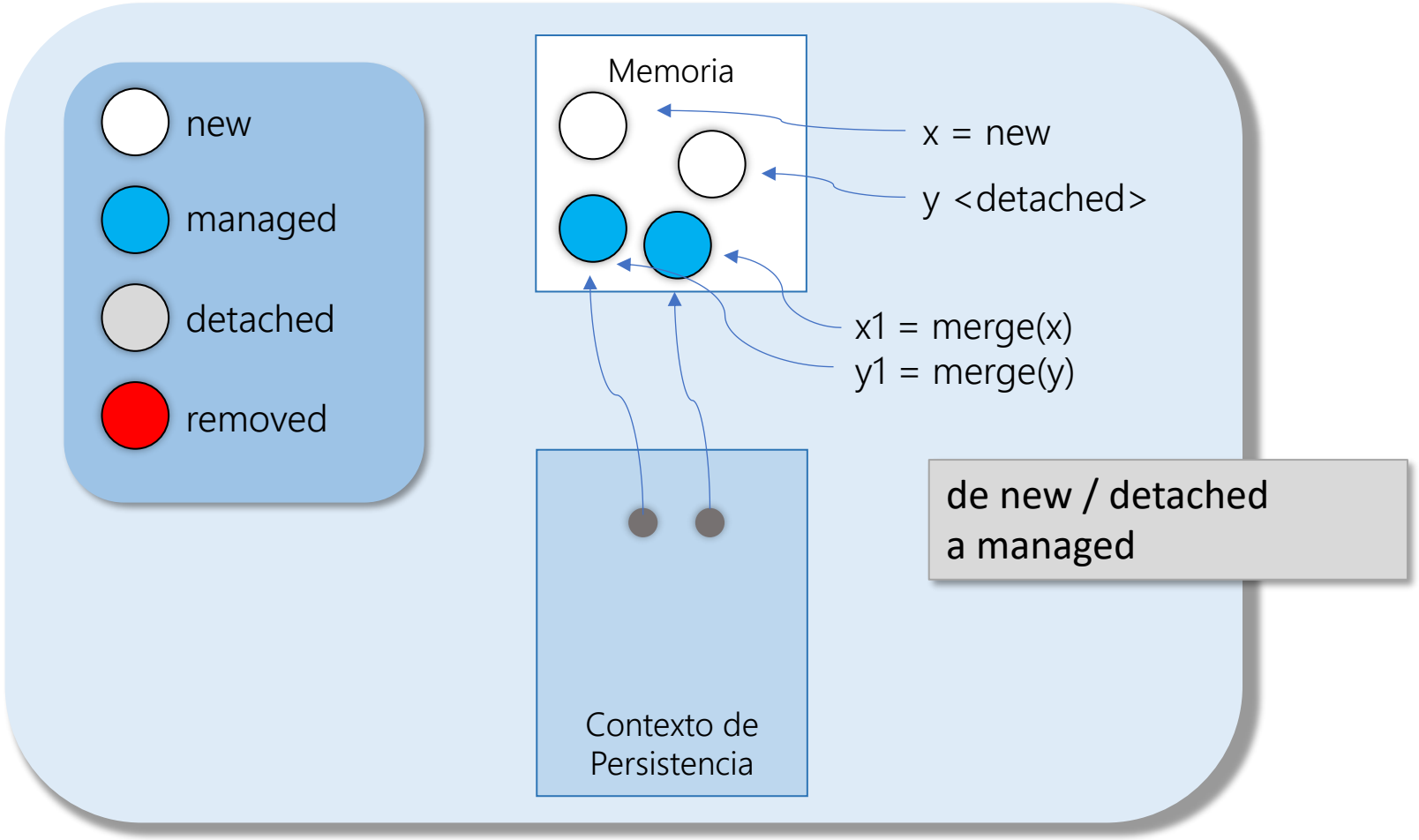
persist()

Convierte una nueva instancia en persistente



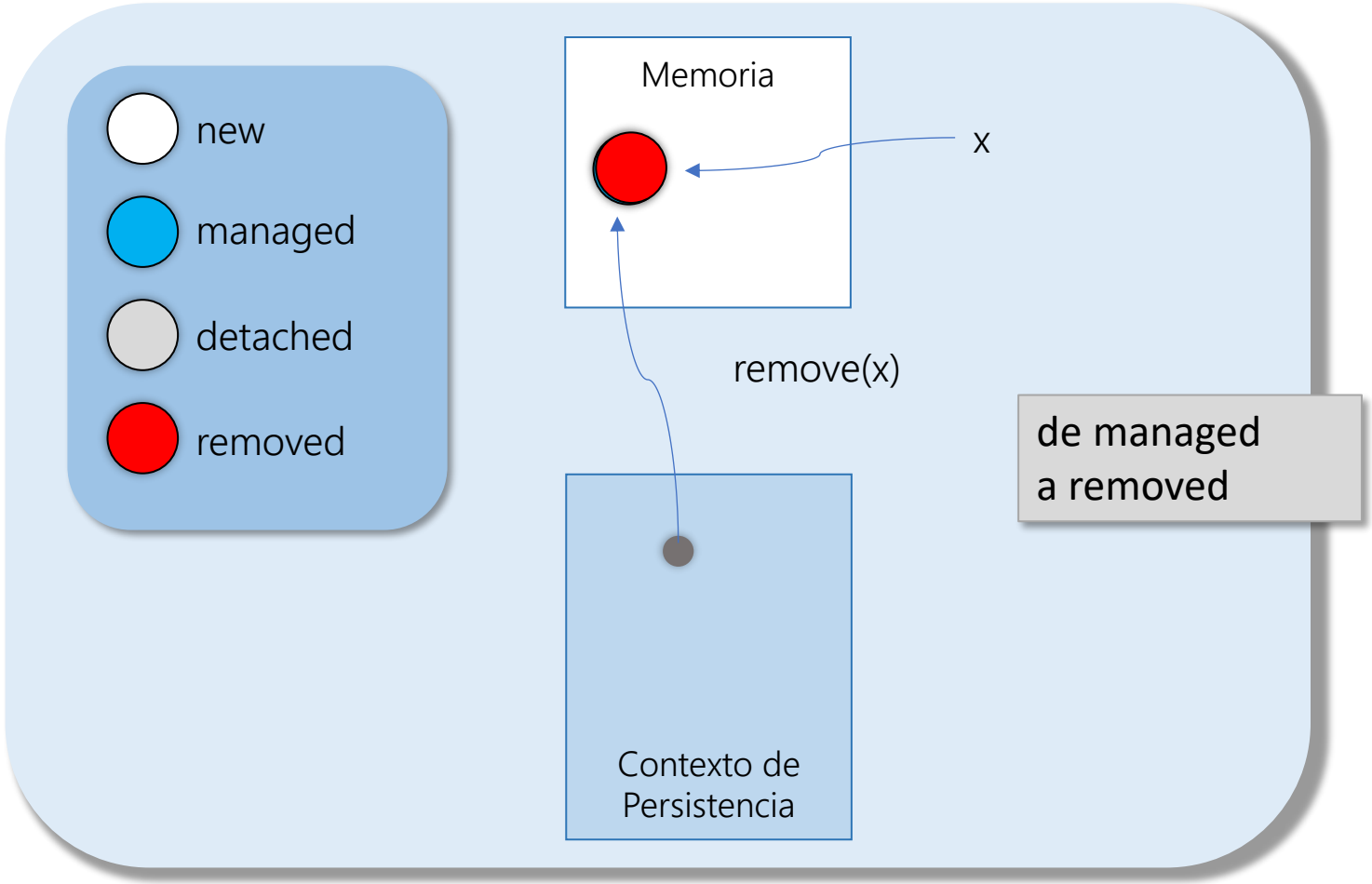
merge()

Combina el estado de una instancia con el contexto



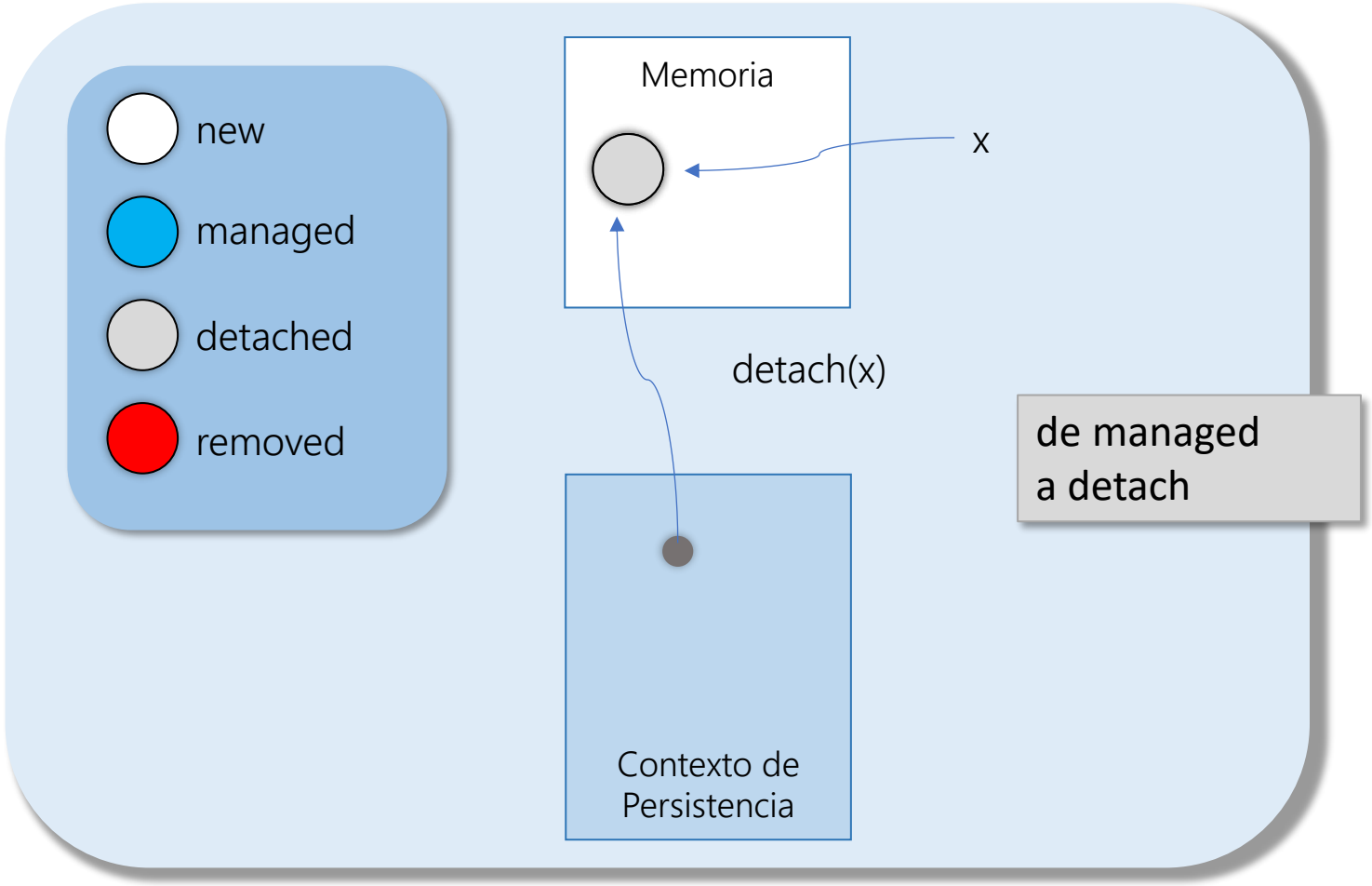
remove()

Pasa el obj de managed a removed

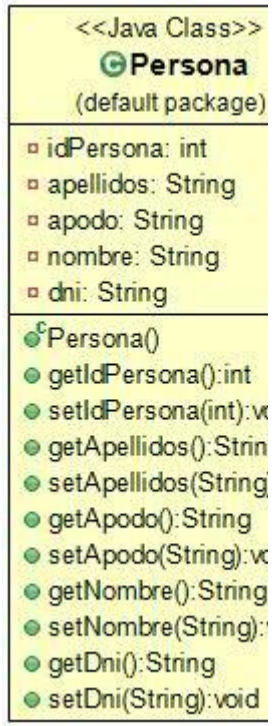


detach()

Desasocia la instancia del contexto



JPA – Si los nombres no coinciden




```java
@Entity
@Table(name="personas")
public class Persona implements Serializable {
@Id
@Column (name = "id_persona")
private int idPersona;

@Column (name = "p_apellidos")
private String apellidos;

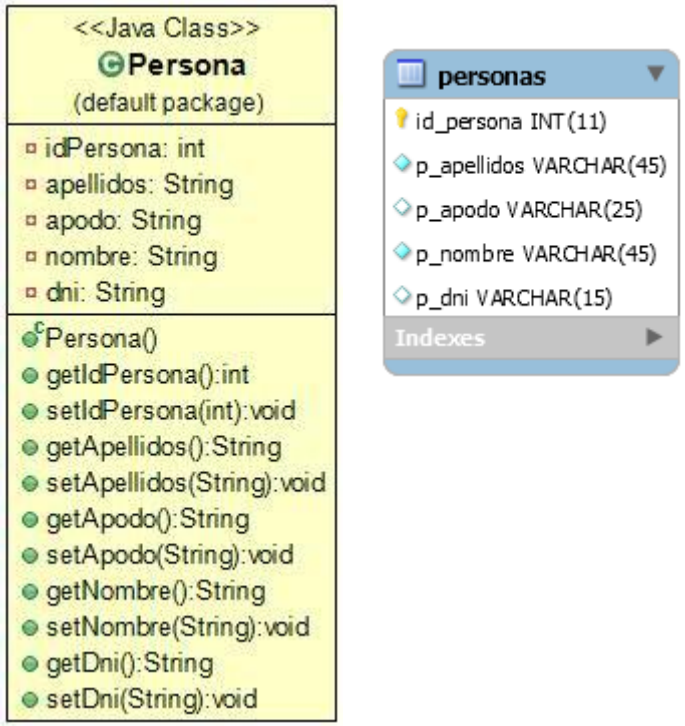
@Column (name = "p_apodo")
private String apodo;

@Column (name = "p_nombre")
private String nombre;

@Column (name = "p_dni")
private String dni;

public Persona() {
}
…
```

JPA – Si los nombres no coinciden




```java
@Entity
@Table(name="personas")
public class Persona implements Serializable {

@Id
@Column (name = "id_persona")
private int idPersona;

@Column (name = "p_apellidos")
private String apellidos;

@Column (name = "p_apodo")
private String apodo;

@Column (name = "p_nombre")
private String nombre;

@Column (name = "p_dni")
private String dni;

public Persona() {
}
…
```

Otras anotaciones

@Transient
Los atributos transient no se almacenan en la bbdd

```
@Transient
private int edad;
```

@Basic (optional=false)
El atributo no admite nulos

```
@Column (name = "p_dni")
@Basic (optional=false)
private String dni;
```

Tipos enumerados

@Enumerated

```
enum Genero {HOMBRE, MUJER};
…
private Genero sexo;
```

```
@Column (name = "p_sexo")
private Genero sexo;
```

```
@Enumerated
@Column (name = "p_sexo")
private Genero sexo;
```

Almacena el orden del valor
(cuidado con el cambio de orden)

```
@Enumerated (EnumType.STRING)
@Column (name = "p_sexo")
private Genero sexo;
```

Almacena una cadena

Tipos Insertables

No tienen identidad. Serán persistidos dentro de un Entidad u otro insertable.

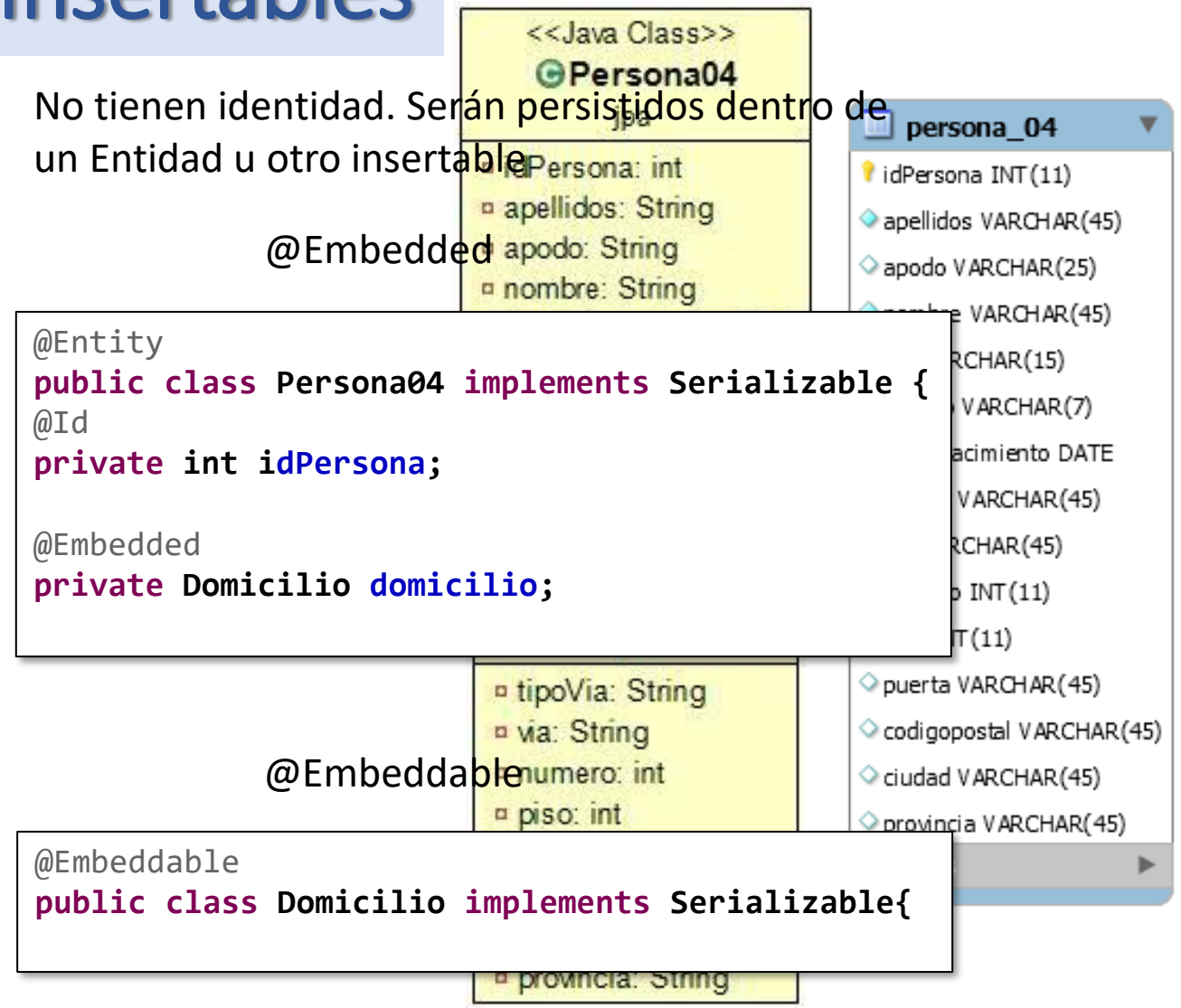
@Embedded

```
@Entity
public class Persona04 implements Serializable {
@Id
private int idPersona;

@Embedded
private Domicilio domicilio;
```

@Embeddable

```
@Embeddable
public class Domicilio implements Serializable{
```



Tipos Insertables

No tienen identidad. Serán persistidos dentro de un Entidad u otro insertable

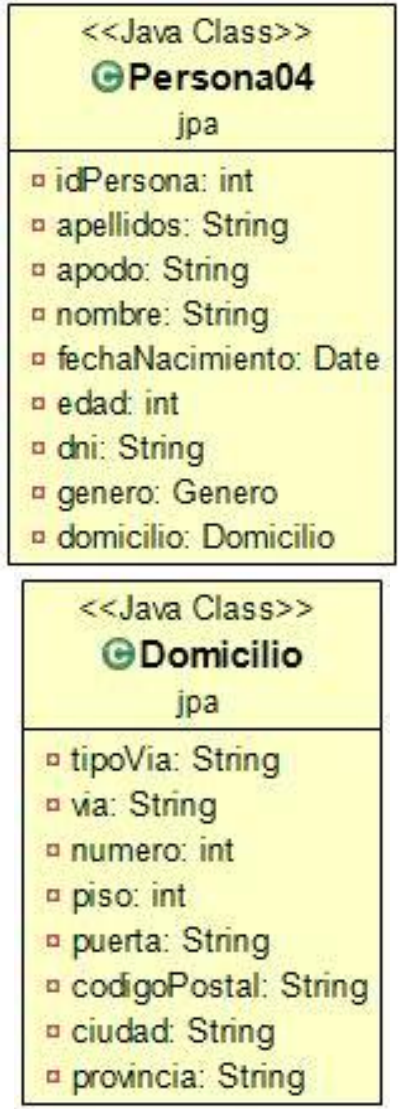
@Embedded

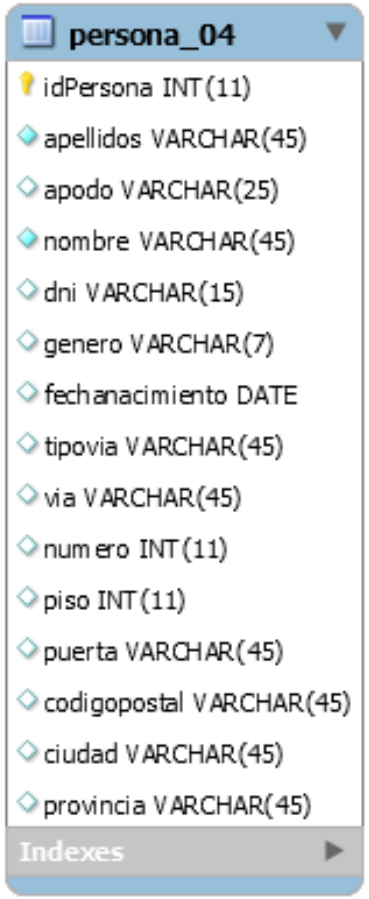
```
@Entity
public class Persona04 implements Serializable {
@Id
private int idPersona;

@Embedded
private Domicilio domicilio;
```

@Embeddable

```
@Embeddable
public class Domicilio implements Serializable{
```





PK Compuestas

- **Debe crearse una clase que represente la PK, con los atributos de la PK Compuesta**

PK COMO INSERTABLE

EmbeddedId property

- La PK compuesta se modela en una clase, que debe ser serializable y definida como @Embeddable
- En la Entidad, el atributo PK se declara con @EmbeddedId

MÚLTIPLES @Id

@IdClass(nombreClaseId.class)

- Multiples atributos @Id en la clase

Tipo de Acceso

Atributos

FIELD

```
@Entity
public class Persona implements Serializable {

  @Id
  private int idPersona;

  private String apellidos;
```

Métodos getters

PROPERTY

```
@Entity
public class Persona implements Serializable {

  private int idPersona;
  private String apellidos;
     …
  @Id
  public int getIdPersona() {
  }
```

Tipos de Acceso heredado

- Las clases insertables heredan el tipo de acceso
- Se puede invalidar (cambiar) el tipo de acceso por defecto o heredado (clases Embeddable), a nivel de clase o a una propiedad en particular
- @Access(AccessType.***PROPERTY)***
- @Access(AccessType.***FIELD)***

Mapeo de Herencia

@Inheritance

- Single Table Strategy

  @Inheritance (strategy=InheritanceType.SINGLE_TABLE)

- Joined Subclass Strategy

  @Inheritance (strategy=InheritanceType.JOINED)

- Table per Class Strategy

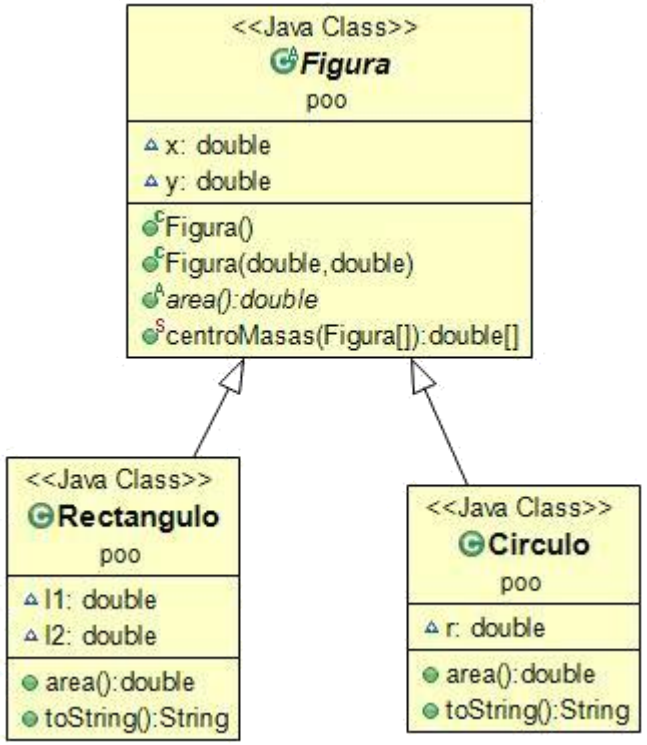
  @Inheritance (strategy=InheritanceType.TABLE_PER_CLASS)

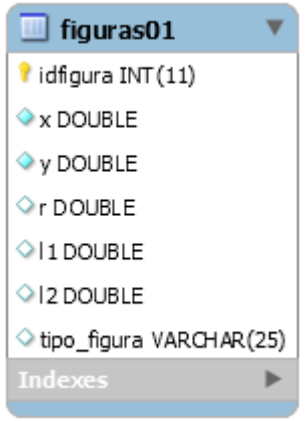
Herencia - Single Table Strategy

@Inheritance (strategy=InheritanceType.SINGLE_TABLE)

Mapeo por defecto.

- Todas las clases de una misma familia son almacenadas en una única tabla.
- Una columna por cada atributo de cada clase y subclase de la familia.
- Una columna adicional para el tipo de clase al que hace referencia cada fila.





Herencia - Single Table Strategy

```java
@Entity
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="tipo_figura", discriminatorType=DiscriminatorType.STRING)
public abstract class Figura {
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
int idFigura;
double x;
double y;
…
```

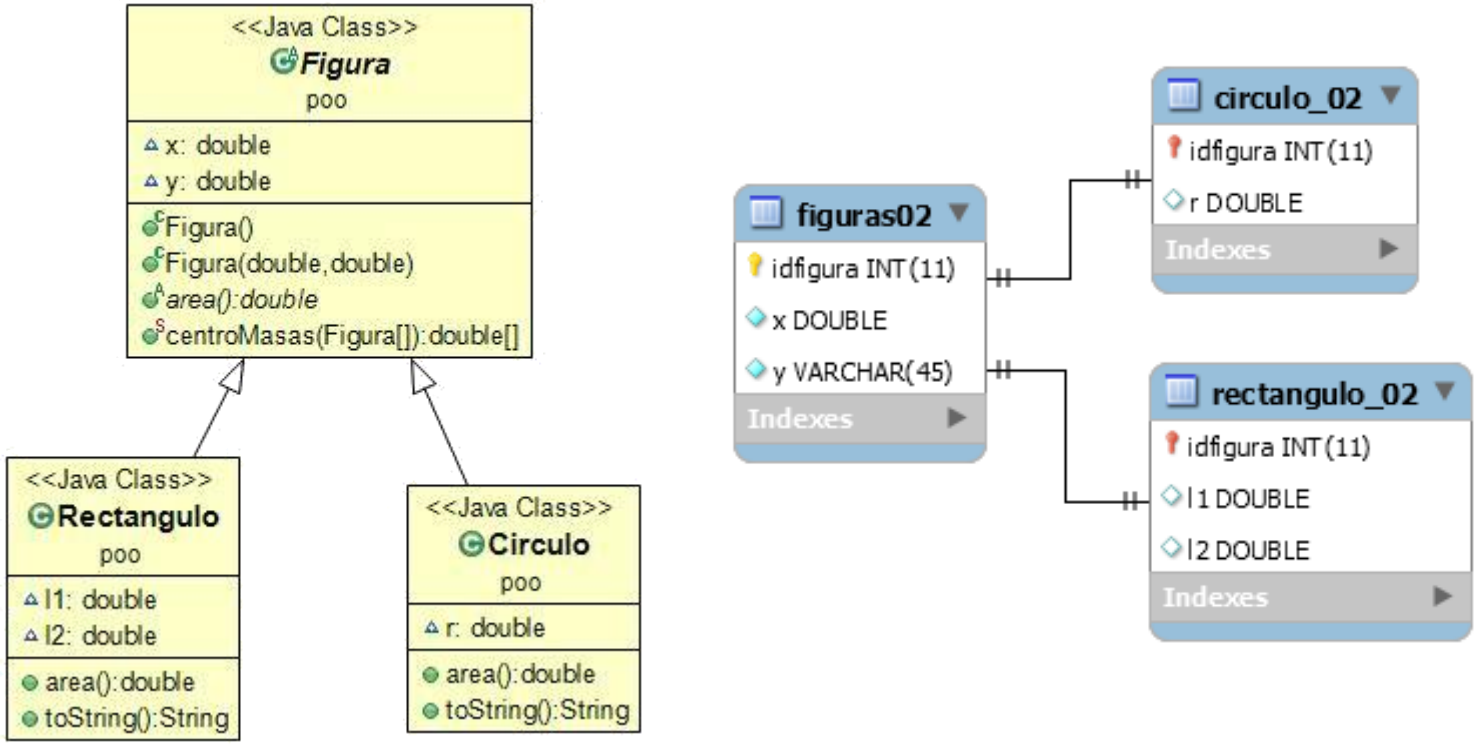
```java
@Entity
@DiscriminatorValue("CIRCULO")
public class Circulo extends Figura {
double r;
…
```

```java
@Entity
@DiscriminatorValue("RECTANGULO")
public class Rectangulo extends Figura {
double l1;
double l2;
…
```

Herencia - Joined Subclass Strategy

@Inheritance (strategy=InheritanceType.JOINED)

- Cada clase y subclase (abstracta o concreta) se almacenará en su propia tabla.
- La clave primaria de la tabla raíz es la misma que la de las subclases.
- Cada subclase almacenará en su propia tabla únicamente sus atributos propios y su clave primaria es a su vez clave foránea a su tabla raíz.



Herencia - Joined Subclass Strategy

```java
@Entity
@Table (name = "figuras02")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Figura02 {

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
int idFigura;
…
```

```java
@Entity
@Table (name = "circulo_02")
public class Circulo02 extends Figura02 {
double r;
```

```java
@Entity
@Table (name = "rectangulo_02")
public class Rectangulo02 extends Figura02 {

double l1;
double l2;
```

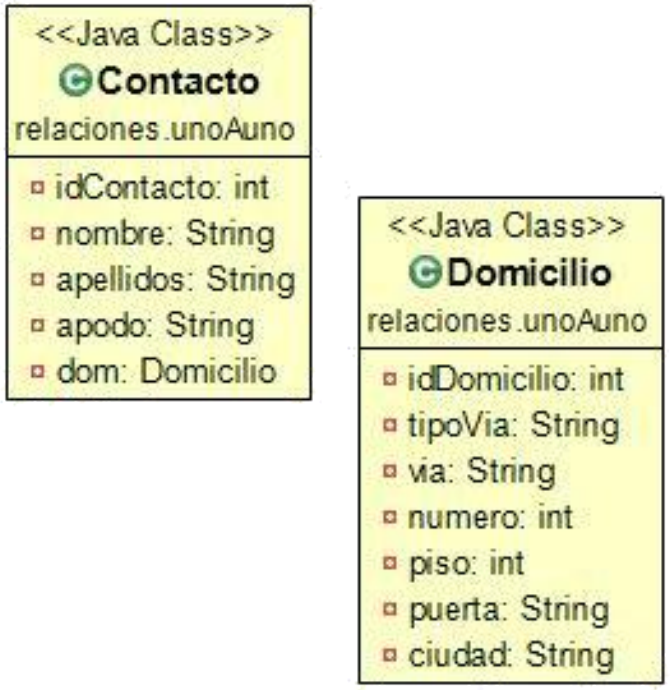
Herencia - Table per Class Strategy

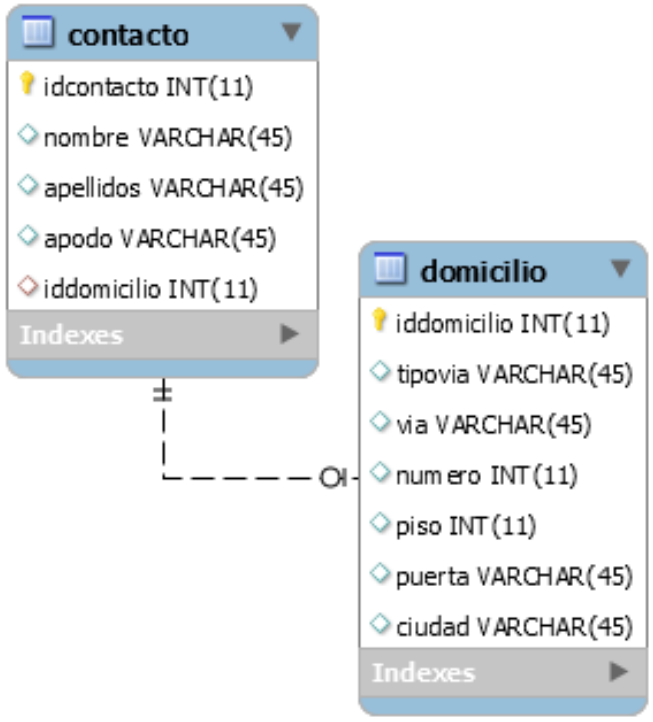
@Inheritance (strategy=InheritanceType.TABLE_PER_CLASS)

- Una tabla por cada clase concreta de la jerarquía.
- Cada tabla incluye todos los atributos necesarios de esa clase, incluidos los heredados.
- Las tablas hijas no comparten la PK con una tabla padre (si la hubiere).
- Suele estar menos recomendada que SINGLE_TABLE o JOINED en muchos casos, porque las consultas polimórficas pueden ser más costosas y el esquema puede duplicar columnas.

Mapeo de Relaciones

Uno a uno





@OneToOne (unidireccional)

```java
@Entity
public class Contacto{
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idContacto;
private String nombre;
…

@OneToOne
@JoinColumn(name = "iddomicilio")
private Domicilio dom;
```

```java
@Entity
public class Domicilio {
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idDomicilio;

private String tipoVia;
…
```

@OneToOne (bidireccional) - @OneToOne

```java
@Entity
public class Contacto2{
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idContacto;
private String nombre;
…

@OneToOne
@JoinColumn(name = "iddomicilio")
private Domicilio dom;
```

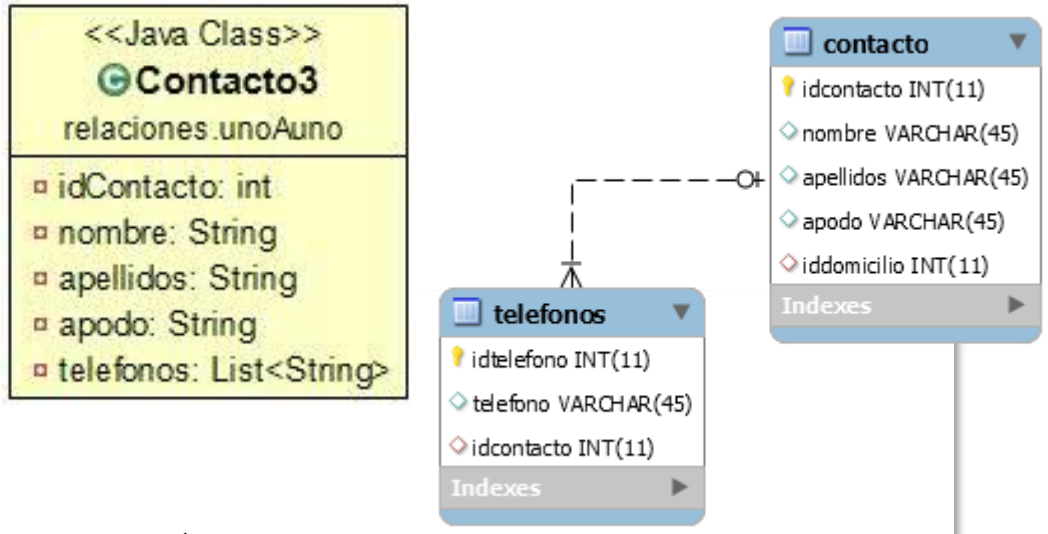
```java
@Entity
@Table (name = "domicilio")
public class Domicilio2 {

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idDomicilio;

private String tipoVia;
…
@OneToOne(mappedBy = "dom")
private Contacto2 contacto;
```

Tipos básicos (Colecciones) - No son muchos a uno

@ElementCollection

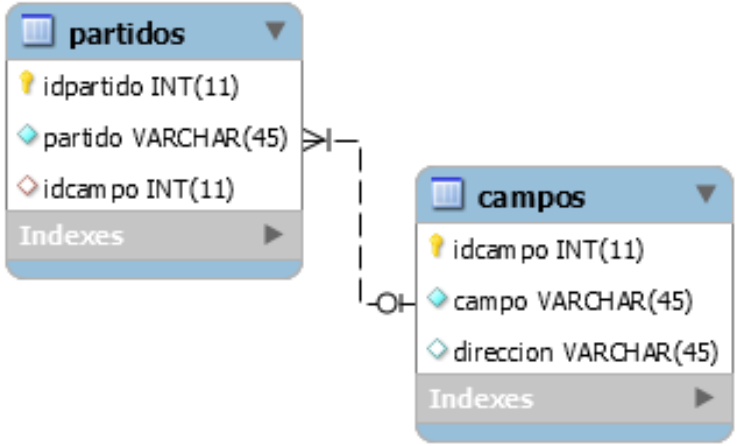



```
@Entity
@Table (name = "contacto")
public class Contacto3{

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idContacto;
private String nombre;
…
@ElementCollection
@CollectionTable (name="telefonos", joinColumns = @JoinColumn (name = "idcontacto"))
@Column(name="telefono")
private List<String> telefonos;
```

@ManyToOne (unidireccional)





@ManyToOne (unidireccional)

```java
@Entity
@Table(name="partidos")
public class Partido implements Serializable {

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idpartido;
private String partido;

@ManyToOne
@JoinColumn (name = "idcampo")
private Campo campo;
```

```java
@Entity
@Table(name="campos")
public class Campo implements Serializable {

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idcampo;
private String campo;
private String direccion;
```

@ManyToOne (bidireccional) - @OneToMany

```java
@Entity
@Table(name="partidos")
public class Partido implements Serializable {

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idpartido;
private String partido;

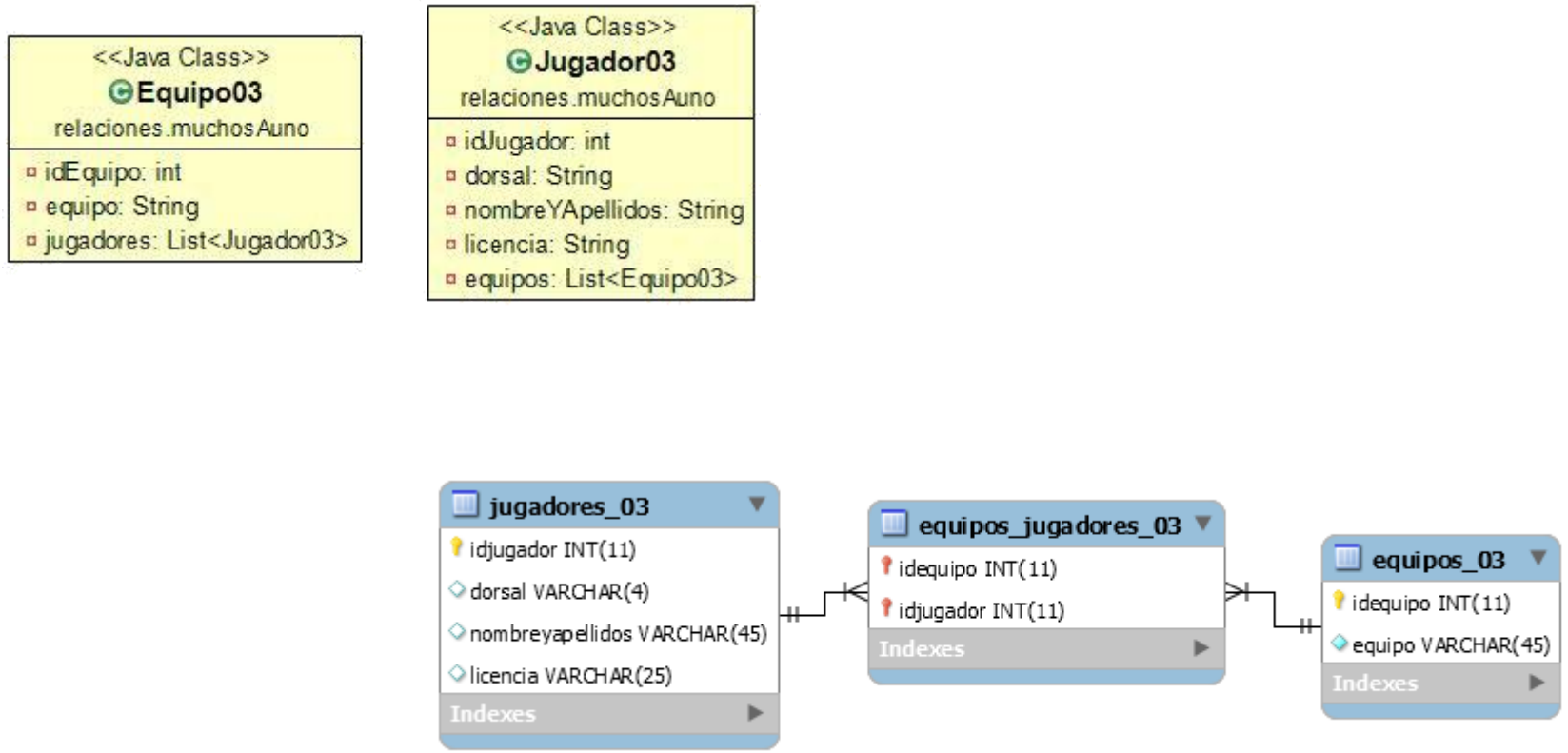
@ManyToOne
@JoinColumn (name = "idcampo")
private Campo campo;
```

```java
@Entity
@Table(name="campos")
public class Campo implements Serializable {

@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idcampo;
private String campo;
private String direccion;

@OneToMany (mappedBy="campo")
private List<Partido> partidos;
```

@ManyToMany


<<Java Class>>
Equipo03
relaciones.muchosAuno
idEquipo: int
equipo: String
jugadores: List<Jugador03>
<<Java Class>>
Jugador03
relaciones.muchosAuno
idJugador: int
dorsal: String
nombreYApellidos: String
licencia: String
equipos: List<Equipo03>
jugadores_03
idjugador INT(11)
dorsal VARCHAR(4)
nombreyapellidos VARCHAR(45)
licencia VARCHAR(25)
Indexes
equipos_jugadores_03
idequipo INT(11)
idjugador INT(11)
Indexes
equipos_03
idequipo INT(11)
equipo VARCHAR(45)
Indexes

@ManyToMany

```
@Entity
@Table(name="equipos_03")
public class Equipo03 implements Serializable {
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idEquipo;
private String equipo;

@ManyToMany
@JoinTable(name="equipos_jugadores_03"
, joinColumns={ @JoinColumn(name="IdEquipo")}
, inverseJoinColumns={@JoinColumn(name="idJugador")}))
private List<Jugador03> jugadores;
```

```
@Entity
@Table(name="jugadores_03")
public class Jugador03 implements Serializable {
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int idJugador;
private String dorsal;
private String nombreYApellidos;
private String licencia;

@ManyToMany(mappedBy="jugadores")
private List<Equipo03> equipos;
```

Lectura Temprana y Demorada – Fetch Type

LAZY - Lectura Demorada

Leerá los datos de la tabla en el momento que quiera utilizarse la propiedad

@ElementCollection
@OneToMany
@ManyToMany

EAGER - Lectura Temprana

Leerá los datos de la tabla al leer el objeto

@OneToOne
@ManyToOne

```
@Basic (fetch = FetchType.LAZY)
private Imagen img;

@OneToOne(fetch = FetchType.LAZY)

@ElementCollection(fetch=FetchType.EAGER)
```

JPQL

- Permite realizar consultas sobre **Objetos** (no tablas)
- Sintaxis similar a SQL

SELECT c FROM Contacto c

```
Query q = em.createQuery("SELECT c FROM Contacto c");
List<Contacto> lista = q.getResultList();
```

- Obtiene todas las instancias de la clase Contacto
- Contacto es el nombre de la clase (no de la tabla)
- c es una abreviatura de la Clase y con él puedo acceder a los atributos de la Clase como si fuera una instancia

SELECT c.nombre FROM Contacto c

```
Query q = em.createQuery("SELECT c.nombre FROM Contacto c");
List<String> lista = q.getResultList();
```

JPQL

- DISTINCT
  - SELECT DISTINCT c.nombre FROM Contacto c
- COUNT()
  - SELECT COUNT (c) FROM Contacto c
- WHERE
  - SELECT c FROM Contacto C WHERE condiciones
- BETWEEN
  - … WHERE c.valor BETWEEN 100 AND 1000
- LIKE (% y _)
  - … WHERE c.nombre LIKE ‘%bravo%’
- ORDER BY
  - SELECT c FROM Contacto c ORDER BY c.apellidos

Queries con parámetros

- **Por posición**
  - …WHERE c.idContato = ?1 AND c.algo = ?2
  - q.setParameter(1, 107)
  - q.setParameter(2, “valor de algo”)

- **Por nombre**
  - …WHERE c.idContato = :id AND c.algo = :algo…
  - q.setParameter(“id”, 107)
  - q.setParameter(“algo”, “su valor”)

Sentencias de actualización

- **UPDATE**
  - UPDATE Contacto c SET c.nombre = ‘Pedro’
    WHERE c.IdContacto =105

- **DELETE**
  - DELETE FROM Contacto c WHERE c.idContacto = 107

JPQL - ejecución

- **createQuery(String jpql) Query**

```
Query q = em.createQuery("SELECT c FROM
Contacto c");
List<Contacto> lista = q.getResultList();
```

- **createQuery(String jpql, Class<T>) TypedQuery**

```
TypedQuery<Contacto> q = em.createQuery(
   "SELECT c FROM Contacto c WHERE c.nombre LIKE
:nombre",
    Contacto.class);
q.setParameter("nombre", "%" + nombre + "%");
List<Contacto> lista = q.getResultList();
```

JPQL – named queries

- **createNamedQuery(String jpql) (Estáticas / Con Nombre)**

```
@Entity
@Table(name = “contactos")
@NamedQuery(name = “Contacto.findAll", query = "SELECT c FROM Contacto c")
public class Contacto implements Serializable {
        …
}

TypedQuery q = em.createNamedQuery(“Contacto.findAll”,
    Contacto.class);
List<Contacto> lista = q.getResultList();
```

JPQL – native queries

```
Query query = em.createNativeQuery("SELECT * FROM CONTACTOS",
Contacto.class);
List<Contacto> lista = query.getResultList();
```

```
Query query = em.createNativeQuery("DELETE FROM CONTACTOS");
int filas = query.executeUpdate();
```